Retrieval Augmented Generation

Ms. Swati Mali

Assistant Professor
Department of Computer Engineering
K. J. Somaiya School of Engineering
Somaiya Vidyavihar University

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Overview

01 **Motivation & Problem Context**

Why LLMs alone are insufficient

02 **What is RAG?**

Definition, architecture, core pipeline

03 **Types & Approaches**

Naive, Advanced, Modular, Agentic RAG

04 **Strengths**

Key advantages over baseline LLMs

05 **Weaknesses & Limitations**

Challenges, failure modes, open problems

06 **Real-World Applications**

Domains and use-case examples

Why Do LLMs Need External Knowledge?

Knowledge Cutoff

LLMs are trained on static datasets with a fixed cutoff date. They cannot access events, publications, or facts that emerged after training.

Hallucination

Without grounding, LLMs confidently generate plausible-sounding but factually incorrect responses — a critical failure mode for high-stakes applications.

Domain Specificity

General-purpose LLMs lack deep expertise in proprietary, specialised, or enterprise-specific knowledge (legal, medical, financial documents).

Lack of Verifiability

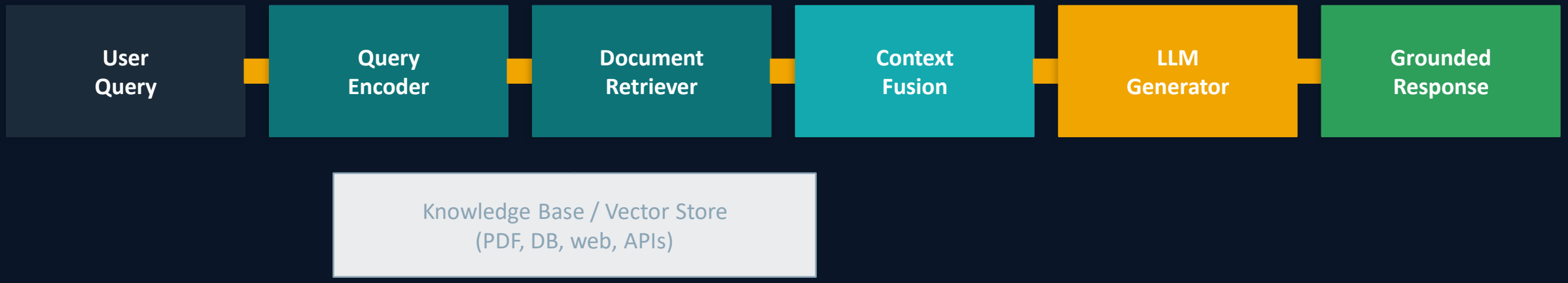
Outputs from standalone LLMs cannot be easily traced to a source, making fact-checking and audit trails nearly impossible.

RAG directly addresses all four limitations by dynamically retrieving relevant, current, verifiable information at inference time.

What is Retrieval-Augmented Generation?

Introduced by Lewis et al. (2020, Facebook AI Research)

RAG is a hybrid AI framework that enhances large language models by coupling them with a **retrieval mechanism** over an external knowledge corpus, enabling the model to produce responses grounded in retrieved, up-to-date, and domain-specific information.



Retriever: Finds relevant chunks from the corpus (dense or sparse)

Generator: LLM that conditions its output on the retrieved context

Knowledge Base: External corpus — static or dynamic, structured or unstructured

Core RAG Architecture

Indexing Pipeline + Retrieval-Generation Pipeline

INDEXING PIPELINE (Offline)

1. **Document Loading**
   Ingest raw sources: PDFs, HTML, databases, APIs
2. **Chunking**
   Split into fixed / semantic / sentence-level chunks
3. **Embedding**
   Encode chunks via embedding model into dense vectors
4. **Vector Store**
   Index vectors in FAISS, Pinecone, Weaviate, Chroma

→

RETRIEVAL + GENERATION (Online)

1. **Query Encoding**
   Embed user query using the same embedding model
2. **Similarity Search**
   Top-k vector search (cosine / dot-product / MMR)
3. **Context Assembly**
   Concatenate retrieved chunks with the original query
4. **LLM Generation**
   Generator produces a final grounded, cited response

Types of RAG: An Evolution

From basic retrieval to autonomous agents

01 Naive RAG

- Simple retrieve-then-read pipeline
- Fixed chunking and top-k retrieval
- Single-pass generation without iteration
- Baseline architecture; easy to implement
- Best for: simple Q&A over static corpora

02 Advanced RAG

- Query rewriting and expansion
- Re-ranking (cross-encoders, Cohere Rerank)
- Hybrid search: dense + sparse (BM25 + DPR)
- Sentence-window & parent-document retrieval
- Best for: complex documents, enterprise search

03 Modular RAG

- Plug-and-play retriever / generator modules
- Iterative & adaptive retrieval loops
- FLARE: forward-looking active retrieval
- RAG Fusion: multi-query generation + RRF
- Best for: research pipelines, custom workflows

04 Agentic RAG

- LLM agent decides when & what to retrieve
- Multi-step reasoning with tool use
- ReAct, Reflexion, IRCOT frameworks
- Self-correcting via feedback loops
- Best for: autonomous assistants, complex tasks

Increasing Complexity & Capability →

Key Retrieval Approaches

Choosing the right retrieval strategy is critical to RAG performance

Sparse Retrieval (BM25 / TF-IDF)

Pros

✓ Fast, interpretable, no training needed, strong for keyword-heavy queries

Cons

✗ Cannot capture semantic similarity; fails on paraphrase or synonym queries

Dense Retrieval (DPR, Contriever)

Pros

✓ Captures semantic meaning; strong for natural language questions

Cons

✗ Requires training or fine-tuning; computationally expensive at scale

Hybrid Search (Sparse + Dense)

Pros

✓ Best of both worlds; robust across query types; used in enterprise search

Cons

✗ Requires score normalisation and careful weight tuning (RRF often used)

Re-ranking (Cross-Encoders)

Pros

✓ Highly accurate re-scoring of retrieved candidates; significantly improves precision

Cons

✗ High latency; cannot scale to full corpora; used as second-stage only

Strengths of RAG

Why RAG is widely adopted in production AI systems

✓ Factual Accuracy & Grounding

By conditioning generation on retrieved documents, RAG dramatically reduces hallucination. The model cites its sources, enabling fact-verification.

✓ No Retraining Required

Knowledge can be updated simply by refreshing the vector store — without expensive fine-tuning or full model retraining cycles.

✓ Domain Adaptability

The same LLM backbone can serve multiple verticals (legal, medical, financial) simply by swapping or partitioning the knowledge base.

✓ Transparency & Explainability

Retrieved passages provide an audit trail. Users and auditors can verify which sources informed a given answer — critical in regulated industries.

✓ Cost-Effective Scaling

Storing knowledge externally is far cheaper than encoding it in model parameters. As knowledge grows, only storage scales — not compute.

✓ Dynamic & Real-Time Knowledge

The knowledge base can be updated continuously, enabling the system to handle breaking news, real-time data feeds, and live enterprise databases.

Weaknesses & Challenges

Known failure modes and active research problems

✗ Retrieval Quality Bottleneck

Generation quality is fundamentally bounded by retrieval quality. Irrelevant or misleading retrieved chunks propagate errors into the final response (GIGO problem).

✗ Context Window Limitations

Only a limited number of retrieved chunks fit in the context window. With long documents, critical information may be lost or diluted.

✗ Retrieval-Generation Mismatch

The retriever and generator are often trained with different objectives. Misalignment between them leads to retrieved content that the generator cannot effectively use.

✗ Multi-Hop Reasoning Failure

Standard RAG struggles when the answer requires synthesising information across multiple non-adjacent documents (multi-hop, multi-document reasoning).

✗ Latency Overhead

Each query requires an embedding computation + vector search + LLM inference. This pipeline introduces significant latency vs. a standalone LLM.

✗ Embedding Model Dependency

The quality of retrieval depends heavily on the embedding model used. Domain mismatch between the embedding model and the corpus degrades performance.

Open Problems & Research Frontiers

Areas where RAG remains an unsolved challenge

01 Faithfulness vs. Creativity Tension

RAG constrains the model to retrieved content, potentially suppressing valid inferences. How to balance grounding with generative flexibility is an open design problem.

02 Chunk Boundary Artefacts

Splitting documents at arbitrary boundaries fractures contextual coherence. Semantic chunking methods (propositions, discourse units) are actively being researched.

03 Evaluation Benchmarking Gap

No standardised benchmark captures the full RAG pipeline. Metrics like RAGAS, RGB, and ARES partially address this, but retrieval and generation evaluation remain coupled.

04 Adversarial Robustness

RAG systems are vulnerable to corpus poisoning: an attacker injecting misleading documents into the knowledge base can systematically corrupt model outputs.

05 Long-Context RAG

As LLMs gain longer context windows (128K+), the role of retrieval shifts. Understanding when retrieval helps vs. hurts at long context is an active empirical question.

06 Multimodal RAG

Extending RAG to images, audio, code, and tables requires cross-modal embedding spaces and retrieval mechanisms that are still maturing.

Real-World Applications of RAG

Deployed across high-value, knowledge-intensive domains

Healthcare & Medicine

- Clinical decision support (EHR + literature)
- Drug interaction Q&A
- Medical coding assistance
- Patient-facing symptom triage

Legal & Compliance

- Contract analysis and review
- Case law research assistants
- Regulatory compliance checking
- Due diligence automation

Enterprise Knowledge

- Internal chatbots over HR / IT wikis
- Customer support over product docs
- Sales enablement (RFP automation)
- Onboarding and training assistants

Education & Research

- Research paper Q&A (Semantic Scholar)
- Adaptive tutoring with curriculum retrieval
- Citation-grounded writing assistants
- Systematic review acceleration

Finance & Banking

- Earnings call Q&A over filings
- Risk report summarisation
- Investment research assistants
- Regulatory capital query systems

Software Engineering

- Codebase-aware code generation (Cursor)
- API documentation Q&A
- Bug triage over issue trackers
- Security vulnerability lookup

RAG vs Fine-Tuning: When to Use Which?

Criterion	RAG		Fine-Tuning
Knowledge Update	**Instant — update vector store**	✓	Requires full retraining
Domain Adaptation	Via knowledge base		**Deep behavioural adaptation**
Factual Accuracy	**High (grounded in retrieved docs)**	✓	Can hallucinate without retrieval
Training Cost	**Low — no model retraining**	✓	High — GPU compute intensive
Latency	Higher (retrieval overhead)		**Lower (single forward pass)**
Interpretability	**High — sources traceable**	✓	Low — knowledge implicit
Long-tail Knowledge	**Excellent — any doc in corpus**	✓	Limited by training coverage
Style / Tone Control	Limited — LLM's base style		**Excellent — trainable**

Note: RAG + Fine-Tuning together (hybrid) often yields the best results in production.

Evaluating RAG Systems

Key metrics and frameworks for measuring RAG quality

Context Precision

RAGAS

What fraction of retrieved chunks are actually relevant to the query?

Context Recall

RAGAS

What fraction of the relevant information was retrieved from the corpus?

Faithfulness

RAGAS / TruLens

Is every claim in the generated answer supported by the retrieved context?

Answer Relevance

RAGAS

Does the generated answer actually address the user's question?

NDCG@k

IR Evaluation

Normalised Discounted Cumulative Gain — classic IR metric for retrieval ranking quality.

Hallucination Rate

FactScore / SAFE

Proportion of generated statements not grounded in retrieved or world knowledge.